

ASCURE Pilot UX Cleanup Plan

Three scoped changes ahead of pilot: capability picker grouping, Site Visit MAINHEAD cleanup, and effective MAINHEAD access preview on the User form. Plan only — no code modified.

Item 1 — Capability picker grouping

Group the existing flat capability checkbox grid into three named fieldsets: **Workspace Access**, **Governance & Reporting**, **Asset Domains**. No schema change. Grouping is driven by a code → group map on the client.

Screens affected

Screen	Where the picker shows up	Behaviour change
Organizations — Create/Edit modal	CapabilityPicker	Replace flat 2-col grid with three labelled fieldsets, each in its own card
Branches — Create/Edit modal	CapabilityPicker	Same
MAINHEADs — Create/Edit modal	CapabilityPicker	Same
Teams — Create/Edit modal	CapabilityPicker	Same
Users — Create/Edit modal	UserCapabilityPicker	Same

The same component is shared across the four enterprise kinds (org/branch/mainhead/team). Users has its own variant. Both need updating.

Files affected

File	Change
apps/admin-web/src/components/enterprise-list-client.tsx	Rework CapabilityPicker (~lines 471–517) to render three grouped fieldsets. Group resolution is a small pure helper.
apps/admin-web/src/components/users-client.tsx	Rework UserCapabilityPicker (~lines 230–276) the same way.
apps/admin-web/src/lib/capability-groups.ts (new)	Single source of truth: capabilityGroupFor(code) → "WORKSPACE" "GOVERNANCE" "ASSET_DOMAIN". Code-prefix map for the 12 seeded codes; unknown codes fall through to ASSET_DOMAIN so contractor-specific additions still render.
apps/admin-web/src/types/enterprise.ts	No type change required (group is derived, not persisted).

Group map (initial — derived from the seed):

WORKSPACE	INSPECTION, MAINTENANCE
GOVERNANCE	QA_VALIDATION, REPORTING
ASSET_DOMAIN	SAVR, SAVT, PENCAWANG, FEEDER_PILLAR, LINK_BOX, CABLE_BRIDGE, UNDERGROUND_CABLE, THERMAL_INSPECTION

Each fieldset carries a one-line caption:

- *Workspace Access* — “Lets users enter this mobile workspace.”
- *Governance & Reporting* — “Cross-MAINHEAD authority and reporting access.”
- *Asset Domains* — “Asset classes this entity is authorised to operate on.”

Estimated effort

Task	Effort
Create capability-groups.ts helper	0.25 day
Rework CapabilityPicker (enterprise)	0.5 day
Rework UserCapabilityPicker	0.25 day
Visual QA across 5 modal contexts	0.25 day
Smoke test pilot ticking scenarios (SAVR+INSPECTION, etc.)	0.25 day
Total	~1.5 days

Migration impact

- **Schema migration:** none.
- **Data migration:** none.
- **API contract:** unchanged. Server keeps returning a flat capabilities[]; grouping is purely a client-side render concern.
- **Rollback:** revert the two picker components; no data side-effects.

Item 2 — Site Visit MAINHEAD cleanup

Remove the “Text MAINHEAD only” path from the Site Visit create form. Require a structured MAINHEAD selection. Preserve legacy text values on existing visits as read-only.

Screens affected

Screen	Behaviour change
Site Visits — list	No behaviour change. MAINHEAD column still uses the existing <code>displayMainhead()</code> fallback so legacy rows continue to render.
Site Visits — Create modal	Remove the “MAINHEAD Text” free-text input. Drop the dropdown’s empty “Text MAINHEAD only” option. MAINHEAD select becomes required . Submit blocked until selected.
Site Visit — Detail page	Existing <code>mainheadRecord?.name</code> → <code>mainheadRecord?.code</code> → <code>mainhead</code> → “Not recorded” fallback stays. When <code>mainheadRecord</code> is null but <code>mainhead</code> (text) is set, badge it as Legacy MAINHEAD so the QA reader knows it’s a pre-cleanup record. No edit affordance.
Mobile (Inspection workspace)	Out of scope for this plan — but flagged: any mobile path that creates Site Visits with <code>mainhead</code> text needs the same gate before this change is promoted. Confirm before rollout.

Files affected

File	Change
<code>apps/admin-web/src/components/site-visits-client.tsx</code>	Drop the “MAINHEAD Text” <code><label></code> block (~lines 847–855). Replace the dropdown empty-option label “Text MAINHEAD only” (~line 807) with “Select MAINHEAD” and mark the <code><select></code> required. Remove <code>mainhead: createForm.mainhead.trim() undefined</code> from the payload (~line 1232). Update <code>DEFAULT_SITE_VISIT_CREATE_FORM</code> to drop <code>mainhead</code> .
<code>apps/admin-web/src/components/site-visit-detail-client.tsx</code>	Wrap <code>displayMainhead()</code> (~lines 85–92) callers so the “Legacy” badge renders when <code>visit.mainheadRecord == null && visit.mainhead</code> is present.
<code>apps/api/src/site-visits/site-visits.service.ts</code>	At ~line 1104 change the validator from <code>if (!dto.mainhead && !dto.mainheadId)</code> to <code>if (!dto.mainheadId)</code> . Error message: “MAINHEAD must be selected. The free-text MAINHEAD field is no longer accepted.” Keep the <code>mainhead</code> column populated on read (existing rows). For new creates, the column is left null and only <code>mainheadId</code> is recorded.
<code>apps/api/src/site-visits/dto/create-site-visit.dto.ts</code>	Change <code>@IsOptional()</code> to <code>@IsNotEmpty()</code> <code>@IsUUID()</code> on <code>mainheadId</code> (~line 46). Keep <code>mainhead?</code> (~line 107) as <code>@IsOptional()</code> for backward compatibility with any in-flight mobile builds — but the server-side validator above blocks it from satisfying the create requirement on its own. After the mobile build is verified to use <code>mainheadId</code> , drop the <code>mainhead?</code> DTO field entirely (follow-up).
<code>apps/api/src/site-visits/dto/list-site-visits-query.dto.ts</code>	No change. The <code>mainhead</code> query parameter is a search filter, not a create input; keep it for legacy row lookup.

Estimated effort

Task	Effort
Admin form changes (site-visits-client.tsx)	0.5 day
Detail page Legacy badge	0.25 day
API DTO + validator change	0.25 day
Reconciliation report — list legacy visits with text-only MAINHEAD that didn't auto-link	0.5 day (script, not a UI; output goes to ops)
Smoke test: create new visit, view legacy visit, list filter, mobile create attempt without mainheadId	0.5 day
Total	~2 days

Migration impact

Aspect	Detail
Schema	No structural change required for pilot. SiteVisit.mainhead column stays nullable; new rows leave it null. A follow-up migration to drop the column can wait until all legacy rows are reconciled.
Data	Existing rows untouched. One-time backfill recommended but not blocking: case-insensitive match SiteVisit.mainhead text to Mainhead.name / Mainhead.code; set mainheadId when uniquely matched. Unmatched rows: leave as-is, surface in the reconciliation report for ops to fix from the detail page (requires a tiny future edit affordance — out of scope here, flag for backlog).
API contract	mainheadId becomes required on POST /site-visits. mainhead body field still parsed but ignored for validation. Breaking change for any mobile build that posted text-only; must confirm mobile uses mainheadId before deploy.
Mobile coordination	Required check. Verify the mobile site-visit-create payload includes mainheadId. If it doesn't, ship the mobile fix first, then the API.
Rollback	Revert the validator and DTO to if (!dto.mainhead && !dto.mainheadId). Admin form changes can be reverted independently.

Item 3 — Effective MAINHEAD access preview on User form

Below the **MAINHEAD Access** and **Region Access** multi-selects, render a computed line showing the resolved set of MAINHEADs the user will actually see. Uses existing data — no new API needed.

Screens affected

Screen	Behaviour change
Users — Create modal	New EffectiveMainheadPreview block immediately below the Region Access picker. Updates reactively as the admin toggles checkboxes.
Users — Edit modal	Same. On first open, pre-populates from the user's existing access rows.
Users — list	No change.
User detail (if added later)	Same component reused. Currently out of scope.

Resolver — client-side

Mirrors steps 1 + 2 of V10's MAINHEAD Visibility Algorithm:

```
direct      = picked mainheadIds
viaRegion   = MAINHEADs whose operationalRegionId ∈ picked regionIds
effective   = direct ∪ viaRegion (deduped, sorted by name)
```

Render as: "Effective MAINHEAD access: KL Timur, KL Barat, Subang (via Region: Klang Valley) + Bentong (direct)". Empty state: "No MAINHEAD access – user will not see any visits."

Why client-side: the mainheads list returned by `/enterprise/options` already includes `operationalRegionId` (confirmed in `apps/api/src/enterprise/enterprise.service.ts:393`), so the admin form has every input it needs. Doing it server-side would require a new `GET /users/:id/mainheads?simulated=...` endpoint — unnecessary churn for the pilot.

Steps 3–7 of the visibility algorithm (team/branch inheritance, legacy user `.mainheadId`, ADMIN override, QA override) are deliberately **not** shown in the preview:

- Team/branch inheritance is fragile and changes as memberships shift — would mislead at the moment of edit.
- ADMIN/QA overrides are blanket; surfacing them would just say "sees everything" for those roles. Show role-based banner instead: "Role: ADMIN – sees all MAINHEADs (override)."

Files affected

File	Change
apps/admin-web/src/components/users-client.tsx	Add EffectiveMainheadPreview component (small, ~50 LOC). Render it inside UserFormModal after the UserAccessPicker for Region Access (~line 486). Subscribes to values.mainheadAccessIds, values.operationalRegionAccessIds, values.role, and the loaded enterpriseOptions.mainheads + enterpriseOptions.operationalRegions.
apps/admin-web/src/lib/mainhead-resolver.ts (new)	Pure helper: resolveEffectiveMainheads({ direct, regionIds, mainheads }) → { direct: Mainhead[], viaRegion: Map<RegionId, Mainhead[]> }. Reused later if we ever surface the same preview on the User detail page or team form.

No API change. No type change. No new endpoint.

Estimated effort

Task	Effort
mainhead-resolver.ts helper + unit thinking	0.25 day
EffectiveMainheadPreview component + role-override banner	0.5 day
Wire into UserFormModal (Create + Edit)	0.25 day
Visual QA: empty, direct-only, region-only, mixed, ADMIN, QA-like role	0.25 day
Total	~1.25 days

Migration impact

- **Schema migration:** none.
- **Data migration:** none.
- **API contract:** unchanged.
- **Server load:** zero — fully client-side.
- **Rollback:** remove the component import; no data side-effects.

Combined effort

Item	Effort
1. Capability picker grouping	1.5 days
2. Site Visit MAINHEAD cleanup	2.0 days
3. Effective MAINHEAD access preview	1.25 days
Cross-cutting smoke pass + brief admin doc note	0.75 day
Total	~5.5 days

Single engineer. No external dependencies beyond mobile confirmation for Item 2.

Rollout order

The order is chosen so each step de-risks the next, with the lowest-blast-radius change first and the only step that touches the API last.

Step	What ships	Risk	Why this order
1. Item 1 — Capability picker grouping	Admin web only. Two component files + one helper.	Effectively zero. Pure visual reshuffle.	Lands the smallest, most reversible change first. Pilot admins immediately benefit when setting up users. Surfaces no API or data concerns, so it can ship to production behind any normal review.
2. Item 3 — Effective MAINHEAD access preview	Admin web only. One new helper + one new sub-component in the user form.	Very low. Client-side computation over already-loaded data.	Independent of Item 2's API change. Improves the user-setup workflow that the pilot will hammer on. Doing it before Item 2 means pilot ops staff have a self-service way to spot misconfigured access during onboarding.
3. Item 2 — Site Visit MAINHEAD cleanup	Admin web + API. Requires mobile compatibility check.	Medium. Breaks any client that posts text-only MAINHEAD.	Ships last because it is the only step that (a) changes server validation and (b) requires mobile coordination. Order of sub-deploy: (i) mobile confirms it sends <code>mainheadId</code> , (ii) deploy API validator change to staging, (iii) deploy admin form change, (iv) run reconciliation script in read-only mode, (v) promote to production. Roll-forward: if the reconciliation report shows >N unresolved legacy text MAINHEADs (operationally chosen threshold), pause and run a manual reconcile session before unblocking new creates.

Per-step exit criteria

- **After Step 1:** All five capability-picker contexts render in three groups. No console errors. Existing capability assignments load and save unchanged.

- **After Step 2:** Editing a user with mixed direct + region access shows the correct effective list and matches what GET /users/me/mainheads returns when that user logs in (manual cross-check).
- **After Step 3:** New site visits cannot be created without mainheadId. Legacy visits still display their text MAINHEAD with the Legacy badge. Reconciliation report exists and lists every site visit with mainhead text but null mainheadId. Mobile create flow verified end-to-end against staging.

Out of scope (logged for backlog, not in this plan)

- Adding a Capability.kind enum column (item B in the prior assessment).
- Replacing the Organization type placeholder enum with the Blueprint vocabulary.
- Adding QA persona roles (QA_SUPERVISOR, QA_INSPECTOR).
- Marking MAINHEAD's "Legacy Branch" block read-only.
- Dropping the silent user.mainheadId mirror.
- Dropping the SiteVisit.mainhead column entirely (do after all legacy rows reconciled).
- Adding a GET /users/:id/mainheads admin endpoint (only if we later want server-side preview parity).
- Editing MAINHEAD on existing site visits from the admin detail page (needed to fully resolve unmatched legacy rows).

These can be sequenced after the pilot stabilises. None of them block Items 1–3.